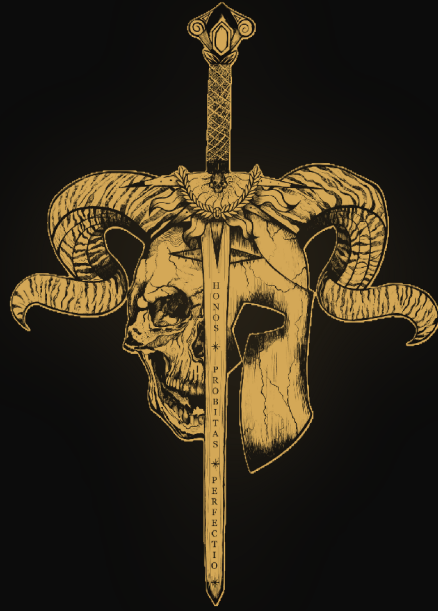




HONOS · PROBITAS · PERFECTIO

DOCUMENTO DE BUILD

APLICACIÓN MANLAB · PWA

Arquitectura · Stack · Flujo de pago · Modelo de datos del Reto · Integración
del Clon · Contrato de API Versión 2.0 · Confidencial · Para: desarrollador
contratado Decisiones de Dirección cerradas (Anexo A). Documento interno
— no reproducir fuera del equipo de desarrollo.

CÓMO LEER ESTE DOCUMENTO

Está escrito para que un desarrollador profesional pueda construir la app **completa** sin reuniones de aclaración previas. Cada sección dice *qué* construir y *por qué*. Las decisiones de negocio están **cerradas** (Anexo A).

Este documento es el **maestro narrativo**. El paquete de handoff trae además los artefactos que arrastras al repo: `schema.sql` (esquema ejecutable, fuente de verdad), `api-contract.md` (endpoints), `.env.example` (variables), `interfaces.ts` (tipos + `VerdictProvider`), `verdict-system-prompt.txt` (prompt verbatim), `verdict-fewshot-examples.md`, `migration-spec.md` y `acceptance-checklist.md`. Los archivos pesados y credenciales se listan en §14 y §15.

1. RESUMEN EJECUTIVO

ManLab es una marca de desarrollo masculino en español (~550K seguidores, 100% orgánico desde TikTok). Vende suscripciones digitales y productos one-shot. El problema central que esta app resuelve **no es vender más, es retener**: el suscriptor del Clon AI dura en promedio **2.66 meses** y se cae en la transición del **mes 2 al 3**, porque el dolor que lo trajo se alivia antes de que la transformación de identidad esté completa.

La app concentra en un solo lugar todo el ecosistema (Clon conversacional, Reto 100 de 100, videoteca, libros, audiolibros) detrás de **una sola suscripción**. La pieza diferenciadora (el **MOAT**) es que el sistema **lee la bitácora diaria del usuario en el Reto y le devuelve un veredicto en la voz de Master**, conectando los frentes que está fallando (circuito cerrado). Eso convierte el churn pasivo en un compromiso activo diario.

Alcance por fases (no se construye todo de golpe): - **Fase 1 — Corazón de retención**: Auth + suscripción + Reto con bitácora + Veredicto del Clon + Consejo del día + Notificaciones. - **Fase 2 — Lectura**: Lector de libros (EPUB) + reproductor de audiolibros. - **Fase 3 — Comunidad y catálogo**: Videoteca completa + Hermandad/accountability + Rangos + superficie de win-back.

El Clon conversacional (Delphi) puede integrarse desde Fase 1 vía embed, porque ya existe; el **Veredicto (MOAT)** es lo que se construye nuevo en Fase 1.

2. DECISIÓN DE ARQUITECTURA DEL CLON ⚠ LEER ANTES DE COTIZAR

Es la decisión técnica más importante del proyecto: define costo recurrente y complejidad.

El Clon de Master vive hoy en **Delphi**, plan **Scaler (\$399 USD/mes)**, handle `delphi.ai/manlab` (~82.3K "mind"). Delphi es una plataforma de clon conversacional de terceros.

Hallazgo crítico: en el plan Scaler, Delphi ofrece embed en sitio web, marca propia (white-label), dominio personalizado, 10 ubicaciones de embed, 20 "Actions" y sync con CRM. **NO ofrece API externa, "Custom Applications" ni SSO**: esos tres son exclusivos del plan superior (Immortal, precio por contacto de ventas, nivel enterprise).

Consecuencia: el MOAT —que el Clon lea la bitácora del usuario— **no se puede lograr inyectando datos dentro de Delphi en el plan actual**, porque eso requiere la API (Immortal). Por lo tanto la arquitectura adoptada es **DESACOPLADA**:

Función	Motor	Por qué
Conversación abierta con el Clon ("platicar con Master")	Delphi embed (iframe, plan Scaler actual, white-label ManLab)	Delphi ya está entrenado con el corpus de Master; hace esto excelente; ya está pagado.
Veredicto del Clon sobre la bitácora (EL MOAT)	API de LLM directa (Anthropic Claude u OpenAI), con system prompt del corpus de Master	Tarea estructurada (entrada: bitácora; salida: diagnóstico en voz de Master). Más controlable, independiente del plan, costo por centavos.

Implicaciones para el desarrollador: - El Clon conversacional se monta como **iframe embebido** (`embed.delphi.ai`), envuelto por el muro de la app (solo usuarios con suscripción activa lo ven). El gating de acceso lo hace la app, **NO Delphi** (no hay SSO en Scaler). - El Veredicto es un servicio propio en el backend que llama a una API de LLM. Ver §8.2 para el system prompt completo, payload y guardrails. - **Voz:** ambos motores deben "sonar a Master". Delphi lo logra por entrenamiento; el Veredicto se calibra con el system prompt y ejemplos few-shot del corpus. Son dos motores distintos, pero el usuario no debe percibirlos como personajes diferentes. **El Veredicto respeta TODAS las reglas de marca del §11.**

Nota — opción futura "Immortal" (decisión ya tomada: vamos por la ruta desacoplada). Si en el futuro Dirección subiera a Delphi Immortal, el Veredicto podría moverse a la API de Delphi (voz unificada) **sin reescribir la app**, cambiando solo el adaptador `VerdictProvider` (§8.4). El documento se construye para que ese cambio sea un swap de proveedor, no una reescritura.

3. ARQUITECTURA TÉCNICA GENERAL

PWA cliente (instalable, offline-friendly para lectura) + backend propio + base de datos + servicios externos.

```
CLIENTE — PWA (instalable, iOS/Android/desktop)
Reto · Bitácora · Veredicto · Clon (iframe) · Lector ·
Audiolibros · Videoteca · Consejo del día · Hermandad
Service Worker (cache lectura) · Push (OneSignal)
```

HTTPS / JWT de sesión propia

```
BACKEND ManLab (API REST/JSON)
Auth · Entitlements (suscripción) · Reto engine ·
Verdict service —→ LLM API (Claude/GPT)
Admin panel API · Notif orchestrator
```

```
DB
(PG)

Stripe
webhook

Bunny/
Mux

OneSignal
push

Delphi embed
(iframe)
```

(cobro en web manlabproject.com)

Cobro fuera de la app (modelo Netflix): la suscripción se compra en manlabproject.com vía Stripe. La app nunca procesa pagos (cero billing de tienda, cero comisión Apple/Google). Detalle en §6.

4. STACK RECOMENDADO

El desarrollador puede sustituir piezas equivalentes; esta combinación está elegida por costo-beneficio, despliegue en cualquier país de LATAM, y por encajar con los activos existentes.

Capa	Recomendado	Alternativa	Notas
Frontend PWA	React + Vite + TypeScript, instalable (manifest + service worker vía Workbox)	Next.js (si SSR)	Debe pasar criterios de "installable PWA". Tailwind.
UI kit	shadcn/ui + Tailwind	cualquiera	Tema oscuro de marca (§12).
Backend	Node.js (NestJS o Express) + TypeScript	Python (FastAPI)	Mismo lenguaje que el front facilita mantenimiento por un solo dev.
Base de datos	PostgreSQL (Supabase o Neon administrado)	MySQL	Supabase aporta auth + storage + Postgres en uno.
Auth	Email + contraseña + magic link; JWT propios	Supabase Auth	NO usar SSO de Delphi (no disponible en Scaler).
Hosting video	Bunny Stream (más barato) o Mux (más features)	Cloudflare Stream	Signed URLs para proteger ~400 videos.
Push	OneSignal	—	Panel admin para envíos manuales (§9).
LLM (Veredicto)	Anthropic Claude (API)	OpenAI GPT	System prompt en §8.2.
Clon conversacional	Delphi embed (Scaler)	—	iframe <code>embed.delphi.ai</code> .
Email transaccional	Brevo (ya en uso)	Resend/Postmark	Recuperación, recibos, win-back, dunning.
Pagos	Stripe (en web ManLab)	—	+ Wise/USDT manual para países sin Stripe (§6.4).
Hosting app/back	Vercel (front) + Railway/Render/Fly (back)	VPS	Económico, escala por uso.

5. IDENTIDAD Y REGLAS DE PRODUCTO (RESUMEN; DETALLE EN §11 Y §12)

- **Tema:** oscuro, sobrio, masculino. Paleta: crema `#F5F0E8`, negro `#0C0C0C`, oro `#D4A857`, oro oscuro `#A47E38`, gris `#8C8780`.
- **Tipografías:** Bebas Neue (impacto/títulos), Inter (cuerpo digital), Cormorant Garamond (display editorial).
- **Voz del producto:** directa, confrontativa, digna. NUNCA registro de autoayuda, NUNCA validación blanda. La app es un espejo del estándar, no una porra.
- Reglas de marca de cumplimiento obligatorio en todo copy in-app: **§11**.

6. MODELO DE COBRO (NETFLIX) – FLUJO DE PAGO EXTERNO

Principio: el dinero entra por la web (`manlabproject.com`) vía Stripe (~3%). La app solo **verifica** si el usuario tiene derecho (entitlement) y desbloquea. Cero cobros dentro de la app.

6.1 Estados de derecho. Cada usuario tiene `entitlement_status` : `active` , `past_due` , `canceled` , `none` . La app desbloquea TODO el contenido si y solo si `entitlement_status = active` .

6.2 Flujo de alta (happy path). 1. El hombre llega a la web ManLab (desde TikTok, etc.) y compra en una Stripe Checkout Session (o Payment Link) hospedada en la web. 2. Stripe crea el `customer` y la `subscription` . Se captura el **email** como llave de identidad. 3. Stripe dispara `checkout.session.completed` y luego `customer.subscription.created/updated` → endpoint webhook del backend ManLab. 4. El backend crea/actualiza `subscriptions` y pone `entitlement_status = active` para ese email. **Si no existe user con ese email, guarda el derecho en `pending_entitlements`** (ver 6.7). 5. El backend envía (Brevo): "Tu acceso a la app ManLab está listo" con link de instalación de la PWA y, si es nuevo, set-password / magic link. 6. El hombre instala la PWA, inicia sesión → `GET /me/entitlement` → `active` → desbloquea.

6.3 Ciclo de vida. - `invoice.paid` → mantiene `active` . - `invoice.payment_failed` → `past_due` ; dispara **dunning** (Brevo). *Hoy hay 92 pagos fallidos por falta de dunning — configurarlo es requisito.* - `customer.subscription.deleted` → `canceled` ; la app bloquea al expirar el periodo pagado. - **Sin periodo de gracia.** Al fallar o vencer, el acceso se corta **al cierre del mismo día** (`entitlement_status` → `none` en el job nocturno). *Decisión de Dirección: el estándar no consiente atrasos.* - **Avisos previos (solo para vencimientos conocidos):** renovaciones programadas (anual) y pagos manuales Wise/USDT → avisar **7 y 3 días antes** (Brevo). Fallo sorpresivo de tarjeta en Stripe no se puede avisar con anticipación: **correo de recobro inmediato** + corte al cierre del día.

6.4 Países sin Stripe (Wise / USDT). El pago se concilia manualmente y un admin otorga el derecho (`POST /admin/entitlement/grant` con email + expiración). El sistema lo trata igual (`source = manual`). Recordatorio de expiración automático 7 y 3 días antes.

6.5 Precio y planes (CERRADO). Una sola suscripción desbloquea TODO. Todos los montos en **USD**. Precios como **config de servidor** (no hardcodear; tabla `plan_config`):

SKU	Precio	Notas
<code>plan_mensual</code>	\$39 USD/mes	Estándar. +30% sobre el Clon solo; mensaje: "\$60+ de valor por \$39".
<code>plan_anual</code>	\$349 USD/año (~\$29/mes, 25% off)	Destacado al suscribirse. Defensa estructural contra el churn del mes 2→3: convierte un cliente de ~2.66 meses en uno de 12 prepago.
<code>plan_fundador</code>	\$29 USD/mes, congelado de por vida	NO público. Solo: (a) win-back de los 1,979 expirados de UDH; (b) migración de la base actual del Clon. <code>price_locked = true</code> .

Reglas: destacar visualmente el anual en el checkout; migrar la base del Clon a `plan_fundador` ; entregar `plan_fundador` por link/código privado a la lista de expirados; el fundador mantiene su precio aunque cambien los públicos.

6.6 Seguridad de pago (obligatorio). El desarrollador NO maneja datos de tarjeta (todo vía Stripe Checkout/Elements). Verificar la **firma del webhook** (`Stripe-Signature`) en cada evento. **Idempotencia** en el handler (Stripe reintenta).

6.7 Caso de borde — email distinto (v2.0, antes faltaba). Si el hombre paga con un email y se registra con otro, sin manejo queda fuera y genera soporte. Solución: el pago que no encuentra user se guarda en `pending_entitlements` (no se pierde) y se materializa al registrarse con un email que coincida; si difiere, Admin usa `POST /admin/entitlement/relink` . Detalle en `migration-spec.md` .

7. MODELO DE DATOS

PostgreSQL. **Fuente de verdad ejecutable:** `schema.sql` . Resumen de tablas:

- `users` — identidad (email CITEXT único), `role` (`user` / `admin`), `onesignal_player_id` .
- `subscriptions` — `source` (`stripe` / `manual`), `plan_code` , `price_locked` , IDs de Stripe, `status` , `current_period_end` . *El entitlement se deriva de la subscription vigente.*
- `pending_entitlements` — derechos de pago sin user todavía (caso email distinto).
- `reto_enrollments` — inscripción al Reto (edición HIERRO, `start_date` , `day_index` 1..100, `status`).
- `reto_daily_logs` — una fila por día: 5 frentes booleanos + `note` libre + `is_complete` GENERADO (los 5 en true).
- `reto_verdicts` — veredictos generados (texto en voz de Master, `weak_link` , `trigger` , costo LLM).
- `books` / `audiobooks` / `videos` / `daily_tips` — contenido.
- `accountability_pairs` / `ranks` / `content_progress` — hermandad, rangos, progreso.
- `notifications_log` / `dunning_events` / `plan_config` — operación y observabilidad.

7.1 Reto 100 de 100 — los 5 frentes (FIJOS, no renombrar): `intelectual` , `espiritual` , `fisico` , `economico` , `social_atraccion` (etiqueta visible "Social / Atracción" — usar "atracción", **nunca** "seducción", §11).

7.2 Lógica del Reto (reglas de negocio — backend): - **a) Calificación 100%-o-nada.** Un día cuenta como cumplido solo si los 5 frentes están en `true` . No hay crédito parcial. La UI comunica sin suavizar: "4 de 5 = día caído". No es autoayuda, es el estándar. - **b) Contador de racha.** `racha_actual` = días consecutivos hasta hoy con `is_complete = true` . Se rompe a 0 con el primer día no completo. Guardar `racha_maxima` . - **c) Eslabón débil (CIRCUITO CERRADO).** Sobre ventana móvil (recomendado: 7 días, configurable), calcular el % de días en `true` por frente. El eslabón débil es el de menor %. La pantalla lo delata visualmente (barra roja, etiqueta "frente que te está hundiendo") y muestra la relación entre frentes. - **d) Doctrina INEVITABILIDAD (visible).** Mensaje fijo, **texto exacto, no parafrasear:** *"Si trabajas diario es inevitable tener resultados; si no, es inevitable fracasar."* - **e) Generación del Veredicto (§8.2).** Se dispara (1) bajo demanda con botón "Pedir veredicto al Clon"; y (2) automáticamente (push) cuando se detecta racha rota o eslabón débil persistente (≥ 3 días fallando el mismo frente).

Nota — análisis de químicos sanguíneos: sigue siendo requisito del Reto, pero se maneja **FUERA de la app** (el participante lo envía al correo de Master y se le da lectura en vivo). NO se construye nada en la app: ni campos, ni subida de archivo, ni recordatorios.

8. INTEGRACIÓN DEL CLON

Dos componentes independientes. **El MOAT es el 8.2.**

8.1 Clon conversacional — Delphi embed. Montar el chat de Delphi como **iframe** apuntando al embed de `delphi.ai/manlab` (origen `embed.delphi.ai`), con marca ManLab (Brand Kit Scaler). **Gating:** el **iframe** solo se renderiza si `entitlement_status = active`; la app no pasa identidad a Delphi (no hay SSO). Ubicación: pestaña "Habla con el Clon" / "Master AI". Resuelve la conversación abierta; **NO** lee la bitácora.

8.2 Veredicto del Clon — el MOAT (servicio propio + LLM API). Toma la bitácora reciente (frentes por día, notas, racha, eslabón débil) y devuelve un veredicto en la voz de Master que conecta los frentes (circuito cerrado) y confronta sin consentir.

Flujo: (1) cliente pide veredicto o se dispara por evento; (2) backend arma el payload con la ventana de 7–14 días; (3) backend llama al LLM con el system prompt de marca + payload; (4) guarda en `reto_verdicts` y devuelve al cliente.

System prompt (fuente de verdad: `verdict-system-prompt.txt`, `verbatim`). Ejemplos few-shot en `verdict-fewshot-examples.md` (PENDIENTE de aprobación de Master). Salida esperada: veredicto de 4–8 líneas que nombra el eslabón débil, conecta el circuito, usa frases firmadas cuando aplica y cierra con `Honos · Probitas · Perfectio`.

8.3 Guardrails técnicos. Temperatura baja-media; tokens cortos. Validar que la salida cierre con la línea doctrinal; si no, reintentar. **Filtro:** si aparece "marco/frame", "seducción" u "omnicracia", rechazar y regenerar (las reglas de marca no son negociables). Registrar costo por llamada. Implementado en `validateVerdict()` (`interfaces.ts`).

8.4 Contrato de interfaz `VerdictProvider` (a prueba de futuro). El Veredicto va detrás de una interfaz para cambiar de proveedor (Claude ↔ GPT ↔ futura API de Delphi Immortal) sin tocar el resto de la app. Definición en `interfaces.ts`. Impl. actual: `ClaudeVerdictProvider`.

9. FUNCIONES POR MÓDULO

- **9.1 Reto con bitácora (Fase 1, núcleo).** Checklist diario de 5 frentes, nota libre, contador de racha, 100%-o-nada, visual del eslabón débil, doctrina INEVITABILIDAD visible, botón "Pedir veredicto al Clon".
- **9.2 Consejo del día (Fase 1).** Tarjeta diaria desde `daily_tips`, editable desde admin. Respeta el Sistema Verbal (§11).
- **9.3 Notificaciones (Fase 1) — qué es real y qué no.** Stack OneSignal + panel admin. **Robusto:** recordatorio diario del Reto (push programado); video nuevo en YouTube (automatizable por webhook/poll → push automático); veredicto automático (racha rota / eslabón ≥3 días). **Manual (botón admin):** video nuevo en TikTok/IG (no hay forma confiable de detectarlo) y aviso de LIVE. **Caveat iOS:** el push en PWA en iPhone funciona pero es menos robusto; ruta de escape en §16.

- **9.4 Live (cerrado).** NO se transmite dentro de la app (caro y frágil). El admin dispara una notificación de cuándo y dónde, con link directo a la plataforma (IG/TikTok/YT/Zoom). Botón manual.
- **9.5 Lector de libros (Fase 2).** Lector EPUB in-app (`epub.js` / `readium`). Convertir los libros (hoy en PDF) a **EPUB reflowable**. Guardar posición (`content_progress`), cache offline.
- **9.6 Reproductor de audiolibros (Fase 2).** Marcadores de capítulo, control de velocidad, posición persistente, fondo/bloqueo. 4 audiolibros listos.
- **9.7 Videoteca (Fase 3).** ~400 videos de UDH detrás del muro, Bunny/Mux con signed URLs. Categorías por módulo. Progreso por video.
- **9.8 Hermandad / accountability (Fase 3).** Parejas que ven la racha del otro y se mandan empujones. Muro simple. Moderación básica desde admin.
- **9.9 Rangos por racha (Fase 3).** Estatus "Hombre en obra" (T2) desbloqueado por racha (`ranks`). En el perfil. **NO gamificación blanda tipo confeti:** es estatus ganado, comunicado con dignidad.

10. PANEL DE ADMINISTRACIÓN

API + UI mínima para que Dirección/Josué opere sin tocar código: push manual (segmentado); aviso de LIVE; aviso manual de video TikTok/IG; editar consejo del día; cargar/editar contenido (libros, audiolibros, videos, rangos); otorgar/revocar/relinicar derecho manual; configurar precios sin redeploy; exportar lista de expirados y disparar win-back; ver dunning (pagos fallidos y estado de recobro). Inventario de endpoints en `api-contract.md`.

11. SISTEMA VERBAL Y REGLAS DE MARCA (CUMPLIMIENTO OBLIGATORIO EN TODO COPY)

Aplican a TODO texto que la app muestre (UI, notificaciones, veredicto, consejo del día). **Violarlas es un bug.**

Sistema Verbal — 5 tiers: - **T1 MASTER:** "Sé ese tipo de hombre". Reservado exclusivamente para cierres doctrinales. Nunca como apertura, título ni subtítulo. Nunca se rota ni se altera. - **T2 ESTRATÉGICO:** "Hombre en obra" (estatus/rangos). - **T3 DOCTRINAL:** "Honos · Probitas · Perfectio" (cierre; la comunidad lo responde al cerrar). - **T4 CATEGORÍA:** "Arquitectura conductual masculina". - **T5 TÁCTICO:** frases que rotan (ej. "La calle no perdona la teoría"). Los recordatorios usan T5 rotando, **nunca T1 como apertura.**

Prohibiciones absolutas: - **OMNICRACIA:** excluido por completo de todo deliverable ManLab. No debe aparecer en ningún lugar de la app. - Nunca "marco" ni "frame" → siempre **"postura"**. - **"Seducción"/"seducir":** restringido al ecosistema premium (libros, web, ESL, MCyP). En la app usar **atracción, magnetismo, presencia, postura.** (El frente 5 se etiqueta "Social / Atracción".) - **Universidad del Hombre:** siempre por su nombre de marca, jamás "OnlyFans". - Personajes canónicos de Master (Vanessa "la Modorrita", Oriane, Karla, Argelia, embajador francés, campamento Fila/Nájera) son del corpus de Master, **no replicables.** No generarlos en copy.

Frases firmadas (autoría de Master, usar tal cual, no atribuir a otros): - "No necesito sentirme bien para hacer las cosas; hago las cosas para sentirme bien." - "Las creencias se rompen con evidencias." - "Tú no eres tu mente, tu mente es tuya."

Cierre/bendición (exacto, preservar donde aplique un cierre mayor):

"Mis príncipes de la creación, mis masculinidades divinas... Vayan con la bendición de ManLab. Mis pasos dejo, mis pasos doy. Tengan la bondad de ser felices."

12. IDENTIDAD VISUAL (DETALLE)

- **Paleta:** crema `#F5F0E8`, negro `#0C0C0C`, oro `#D4A857`, oro oscuro `#A47E38`, gris `#8C8780`.
- **Tipografías:** Bebas Neue (impacto), Inter (cuerpo/UI), Cormorant Garamond (editorial). Incluir como webfonts.
- **Logos (Asset Pack §14):** lockup M+espada, emblema cráneo/cuernos/espada "Honos·Probitas·Perfectio", variantes claras/oscuras, dragón crema.
- **Tono UI:** oscuro por defecto. Acentos en oro, con moderación. **Cero estética de app de hábitos "amable":** es una herramienta de estándar, sobria y seria.

13. FASES DE ENTREGA (ROADMAP Y CRITERIOS DE ACEPTACIÓN)

Criterios testeables completos en `acceptance-checklist.md`. Resumen:

- **Fase 1 — Corazón de retención.** PWA instalable · Auth (email + magic link) · Entitlement vía Stripe webhook · Reto con bitácora completa (5 frentes, racha, eslabón débil/circuito cerrado, INEVITABILIDAD) · Veredicto del Clon (LLM) · Clon conversacional (Delphi embed) · Consejo del día · Notificaciones · Panel admin mínimo · Dunning configurado.
- **Fase 2 — Lectura.** Lector EPUB + reproductor de audiolibros, ambos con progreso persistente y cache offline.
- **Fase 3 — Comunidad y catálogo.** Videoteca (~400, signed URLs) + Hermandad/accountability + Rangos por racha + win-back de los 1,979 expirados.

El pago de cada fase se libera cuando todas las cajas de esa fase están marcadas y validadas por Dirección.

14. ASSET PACK — LO QUE EL DESARROLLADOR DEBE RECIBIR DE DIRECCIÓN

- ☒ **Logos / Brand Kit:** RECIBIDO. Dos kits (lockup + emblema dragón) en PNG (512px y 2000px) y SVG, variantes negro, dorado, oro oscuro, crema y blanco.
- ☐ Tipografías licenciadas (Bebas Neue, Inter, Cormorant Garamond) o confirmación de webfont.

- ☐ Libros cerrados (fuente para conversión a EPUB) — ES e EN.
- ☐ Audiolibros finales (4): M4B/MP3 + marcadores de capítulo.
- ☐ Videos UDH (~400): acceso de descarga/migración y confirmación de que pueden salir de la plataforma actual hacia Bunny/Mux.
- ☐ Banco de "Consejos del día" (o se cargan luego desde el panel).
- ☐ Doctrina del Reto y ejemplos de veredicto aprobados por Master (afinar el few-shot del §8.2 — borrador en `verdict-fewshot-examples.md`).
- ☐ Textos legales: términos, privacidad, política de reembolsos (tasa actual 2.16%).

15. CUENTAS, CREDENCIALES Y SERVICIOS A APROVISIONAR

Servicio	Para qué	Quién provee
Stripe (cuenta ManLab)	Cobro en web + webhooks	Dirección
Delphi (Scaler, ya activo)	Clon embed; confirmar Brand Kit/white-label y origen de embed	Dirección
API LLM (Anthropic/OpenAI)	Veredicto del Clon	Dirección crea key
Bunny Stream / Mux	Hosting de video con signed URLs	Dev configura
OneSignal	Push	Dev configura
Brevo (ya en uso)	Email transaccional + dunning + win-back	Dirección/Dev
Hosting (Vercel + Railway/Render)	App + backend	Dev
Dominio	<code>app.manlabproject.com</code> (confirmado)	Dirección
DB administrada (Supabase/Neon)	Postgres + storage	Dev

16. REQUISITOS NO FUNCIONALES

- **PWA instalable:** manifest válido, service worker, ícono, splash; pasar auditoría Lighthouse PWA.
- **Seguridad:** HTTPS; verificación de firma de webhooks Stripe; signed URLs para video/audio; secrets fuera del cliente (**la key del LLM NUNCA en el front**); rate limiting en el endpoint de Veredicto.
- **Privacidad:** datos mínimos; las notas de bitácora son personales → no exponer en URLs ni logs; acceso restringido por usuario.
- **Rendimiento:** lectura offline cacheada; carga diferida de videoteca.

- **Push iOS — ruta de escape:** si el push PWA en iOS resulta insuficiente, envolver la MISMA PWA en **Capacitor** para push nativo, sin reescribir la app. Diseñar el código pensando en esa envoltura desde el inicio (evitar APIs incompatibles con WebView).
- **Observabilidad:** logs de webhooks, costo por llamada LLM, errores de push.
- **Internacional:** funcionar en cualquier país de LATAM; manejar países sin Stripe (§6.4).

17. DEFINICIÓN DE "TERMINADO" (GLOBAL)

El proyecto se considera entregado cuando: (1) las tres fases cumplen sus criterios; (2) todo copy in-app pasa la revisión §11; (3) el flujo cobro web→app funciona de punta a punta sin intervención manual para Stripe; (4) existe panel admin operable por no-desarrolladores; (5) está documentado el handoff (README, variables de entorno, cómo desplegar) para mantenimiento por un solo desarrollador.

ANEXO A — DECISIONES DE DIRECCIÓN (CERRADAS, V2.0)

1. **Precio:** mensual \$39 USD · anual \$349 USD (25% off, destacado) · fundador \$29 USD/mes congelado de por vida (solo win-back de expirados + migración de base actual del Clon). Todo en USD. (§6.5)
2. **Dominio:** `app.manlabproject.com` (confirmado).
3. **Análisis sanguíneo:** FUERA de la app. No se construye nada (correo + lectura en vivo). (§7.2)
4. **Cobro:** sin periodo de gracia; corte al cierre del mismo día al fallar/vencer. Avisos 7 y 3 días antes en vencimientos conocidos (anual, Wise/USDT); fallo sorpresivo de tarjeta = correo de recobro inmediato. (§6.3–6.4)
5. **Plan anual con descuento + precio fundador:** Sí, ambos. (§6.5)
6. **Ruta del Clon:** DESACOPLADA (confirmada). Delphi Scaler para conversación + LLM para Veredicto. No se sube a Immortal. (§2)
7. **Libros:** conversión a EPUB reflowable para lectura real en móvil. (§9.5)

ANEXO B — GLOSARIO DOCTRINAL MÍNIMO (PARA EL DESARROLLADOR)

- **Circuito cerrado:** los 5 frentes se afectan entre sí; uno caído arrastra a los demás.
- **Inevitabilidad:** "Si trabajas diario es inevitable tener resultados; si no, es inevitable fracasar."
- **Real objetivo del Reto:** programar/controlar la mente — "Que la mente no te diga qué hacer, tú le digas a la mente". (NO es confianza, hábitos ni disciplina por ánimo.)
- **Hombre en obra (T2):** estatus del que está en proceso; base de los rangos.

Honos · Probitas · Perfectio